allowing users to communicate with their devices more effectively. In industrial settings, language models integrated into IoT sensors can facilitate better decision-making by analysing maintenance logs and predicting equipment failures through natural language processing. In healthcare, wearable devices with language models can provide personalised health advice by analysing patient data and responding to spoken queries. Integrating language models in IoT applications enhances real-time data analysis, decision-making, and conversational interactions, improving user experience and operational efficiency.

Running language models for IoT applications can be approached in two primary ways: locally on edge devices or remotely in the cloud or data centre. The choice between local and remote execution of language models for IoT applications depends on the specific requirements and constraints of the use case.

Running language models remotely in the cloud or data centre allows leveraging powerful computational resources to handle complex tasks. This approach can offload the intensive processing from edge devices, enabling the deployment of sophisticated models that may be impractical to run locally. Cloud-based execution supports scalability, accommodating varying loads and larger datasets. It also facilitates model updates and management, ensuring that the latest advancements are readily available. However, this method requires robust network connectivity to transmit data between the edge devices and the cloud, which could introduce latency and potential privacy concerns.

Deploying language models directly on edge devices presents several benefits.

Reduced latency: Processing data locally on edge devices minimises the need for data transmission to remote servers, resulting in lower latency. This is crucial for real-time applications such as voice assistants, where rapid response times are essential for user experience.

Enhanced privacy and security: Keeping data processing at the edge ensures that sensitive information remains on the device, reducing the risk of data breaches and enhancing user privacy and security. This is particularly important in applications involving personal data, healthcare, and finance.

Bandwidth efficiency: Running language models on edge devices reduces the need for constant data transfer between devices and cloud servers. This not only saves bandwidth but also allows for seamless operation in environments with limited or intermittent connectivity. Furthermore, edge devices can operate in remote or low-connectivity areas, providing intelligent services without the dependency on constant internet access.

Challenges of running language models on edge devices

Computational resources: Language models are computationally intensive, requiring significant processing power and memory. Edge devices, typically constrained by hardware limitations, struggle to meet these demands. Ensuring efficient model execution without compromising performance is a critical challenge.

Model size and storage: Language models can be enormous in size, often exceeding gigabytes of memory. Edge devices, with limited storage capacity, face difficulties in storing and loading these models. Techniques such as model compression, quantisation, and pruning are employed to reduce model size while maintaining accuracy.

Power consumption: Edge devices operate on battery power, making energy efficiency a key concern. Running large-scale language models can drain battery life quickly. Optimising models for low-power consumption without sacrificing performance is essential for practical deployment.

Techniques for running language models on edge devices

Model compression: Model compression techniques reduce the size of language models by eliminating redundant parameters and simplifying computations. Pruning, knowledge distillation, and weight sharing are common methods. Pruning removes less important neurons, while knowledge distillation transfers knowledge from a large model to a smaller one. Weight sharing groups similar weights to save memory.

Quantisation: Quantisation involves reducing the precision of model parameters from floating-point to fixed-point representations. This significantly reduces memory requirements and computational complexity. Post-training quantisation and quantisation-aware training are two approaches. While post-training quantisation applies after model training, quantisation-aware training incorporates quantisation during the training process.

Edge-specific model architectures: Designing architectures tailored for edge devices can enhance efficiency. Lightweight models such as MobileBERT and TinyBERT are optimised for resource-constrained environments. These models maintain high performance while reducing computational overhead, making them suitable for edge deployment.

Hardware acceleration: Edge devices can leverage specialised hardware accelerators such as GPUs (graphics processing units), TPUs (tensor processing units), and NPUs (neural processing units). These accelerators are designed to handle AI workloads efficiently, providing substantial performance improvements over general-purpose CPUs.

Use cases of language models on edge devices

Voice assistants: Edge-based voice assistants enable real-time speech recognition and natural language